

Avalon Shelton: A19119684

Iris Li: A19153696

Timothy Carroll: A19112628

Introduction

The purpose of this lab was to practice implementing Sequential Circuits by designing a Sequence Recognizer to react to a SONY TV remote. We used important principles like D-flip flops and shift registers to achieve this. We analyzed the relationships between the inputs and outputs of these functions to get a better understanding of how to build a D-flip flop into a 4-bit shift register, and finally a 12-bit register. We implemented our design through Verilog code and then uploaded the code to the Basys board and connected the board to a breadboard and a receiver to read the information from the remote.

Procedure

- We began by identifying what time delays signified starting, zero and one bits
- Create three Verilog source files. One for the start bit recognizer, zero bit recognizer and the one bit recognizer.
- Because we are utilizing the code from the prelab's 12 bit shift register, the start bit recognizer checks if it starts with a high signal, has 9-10 consecutive low signals, and ends with a high signal
- The one bit recognizer has a longer low period than the zero bit recognizer, and it detects when it has 1's at the start and end, and 4-5 zeros in between
- The zero bit recognizer detects when there's 1's at the start and end of the sequence, and 2-3 zeros in between
- Following the identification we wrote a top module containing a starting recognizer, zero recognizer, and one recognizer. In addition to this we included the 12 bit shift register in order to parse the input signal and create an array based off of the signal at a given time.
- The recognizer worked by identifying the amount of zeroes between each one bit to determine the length of the time delay. We manually calculated the amount of zeroes we should expect by multiplying the time delay by the clock frequency.
- After writing the code, we set up a remote receiver to receive the input signal from the SONY remote (getting rid of the 40khz modulation) and then amplifying it to a level that is compatible with the FPGA.
- To test the remote receiver we attached an oscilloscope to determine whether or not it received signals from the SONY remote.

- We then set up the function generator to simulate the clock cycle, we did this using a 3.3V pk-pk square wave at 3.9 kHz and then routing it to the FPGA
- We then tested multiple cases for the top module to determine if it could properly interpret input signals designated for start, one and zero bits using verilog simulation.
- After determining that the verilog code functioned properly we uploaded the code, and began pin assignment for the FPGA
- Finally connecting the clock signal and the remote receiver circuit to the FPGA, we then tested whether or not the FPGA could properly classify the incoming signals series of start, zero and one bits by corresponding blinking LEDs on the FPGA

Circuit Diagrams / Schematics / Data and Results

Fig 1: Top Module containing 12Shift, one bit, zerobit, and start bit recognizer

```

16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module toptoptopmodule(input data, input clk, input en, input reset, output onebit, output zerobit, output startbit);
24 wire [11:0]q;
25 topmodule12shift segment1( data, clk, en, reset, q[11:0]);
26 onebitrecognizer segment2(q[11:0], onebit);
27 zerobitrecognizer segment3(q[11:0], zerobit);
28 startbitrecognizer segment4(q[11:0], startbit);
29
30
31 endmodule
32
33
34
35

```

Fig 2: zero bit recognizer

```

20 ///////////////////////////////////////////////////////////////////
21
22
23 module zerobitrecognizer(input [11:0]in, output zerobit);
24
25 wire w1,w2;
26 and and1(w1, in[0], ~in[1], ~in[2], in[3]);
27 and and2( w2, in[0], ~in[1], ~in[2], ~in[3], in[4]);
28 or or1(zerobit, w1,w2);
29
30
31
32 endmodule
33
34
35
36
37
38
39
40

```

Fig 3 : start bit recognizer



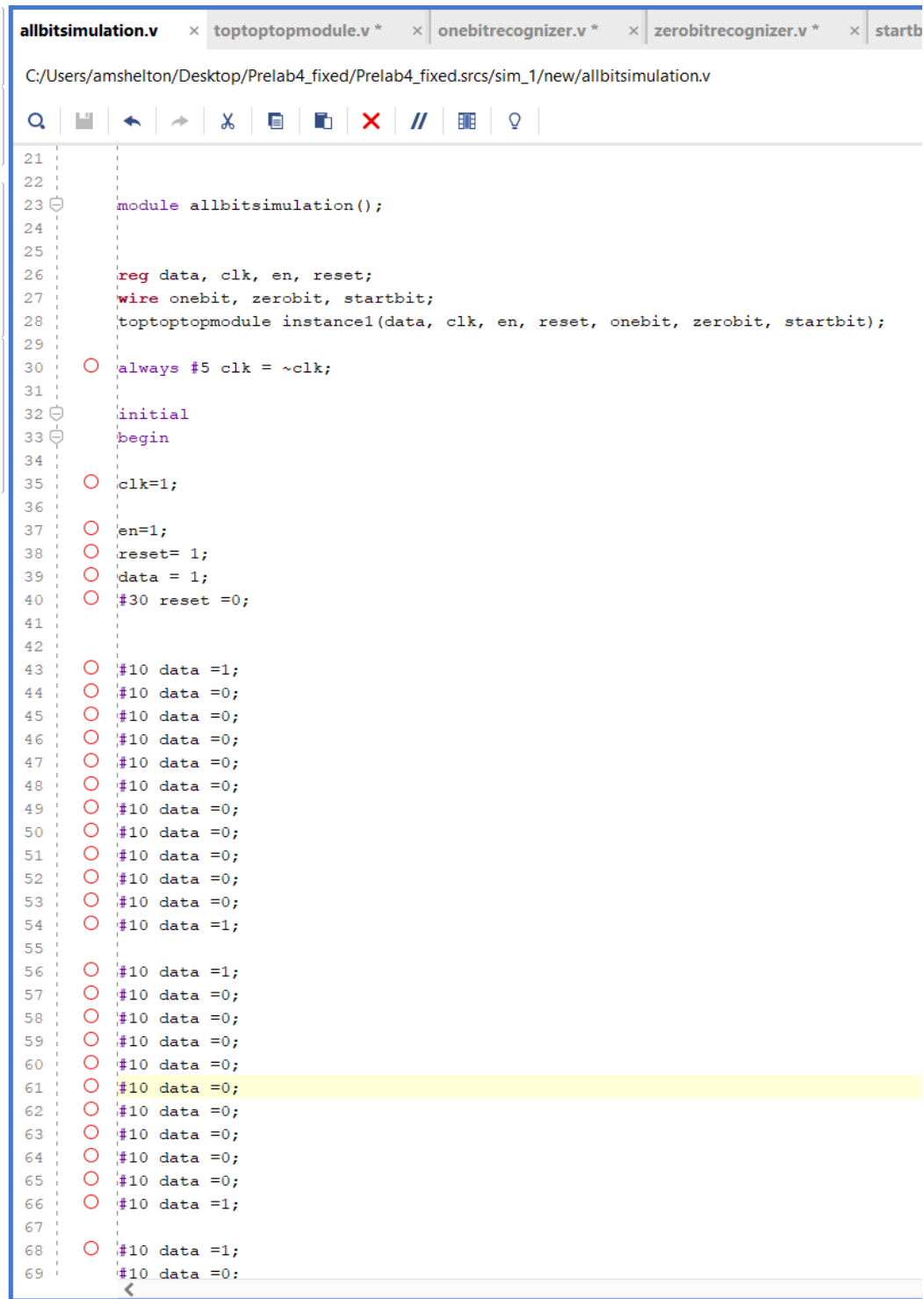
```
20 ///////////////////////////////////////////////////////////////////
21
22
23 module startbitrecognizer(input [11:0]in , output startbit);
24     wire w1,w2;
25     and and1(w1, in[0], ~in[1], ~in[2], ~in[3], ~in[4], ~in[5], ~in[6], ~in[7], ~in[8], ~in[9], in[10]);
26     and and2(w2, in[0], ~in[1], ~in[2], ~in[3], ~in[4], ~in[5], ~in[6], ~in[7], ~in[8], ~in[9], ~in[10], in[11]);
27     or or1(startbit, w1,w2);
28
29
30
31 endmodule
32
33
34
35
```

Fig 4: One Bit Recognizer



```
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module onebitrecognizer(input [11:0]in, output onebit);
24
25     wire w1,w2;
26     and and1(w1, in[0], ~in[1], ~in[2], ~in[3], ~in[4], in[5]);
27     and and2(w2, in[0], ~in[1], ~in[2], ~in[3], ~in[4], ~in[5], in[6]);
28     or or1(onebit, w1,w2);
29
30
31 endmodule
32
33
34
35
36
```

Fig 5: Simulation testing One, Zero, and Start Bits



```
allbitsimulation.v x toptoptopmodule.v * x onebitrecognizer.v * x zerobitrecognizer.v * x startb
C:/Users/amshelton/Desktop/Prelab4_fixed/Prelab4_fixed.srcs/sim_1/new/allbitsimulation.v
Q [Icons]
21
22
23 module allbitsimulation();
24
25
26 reg data, clk, en, reset;
27 wire onebit, zerobit, startbit;
28 toptoptopmodule instance1(data, clk, en, reset, onebit, zerobit, startbit);
29
30 always #5 clk = ~clk;
31
32 initial
33 begin
34
35 clk=1;
36
37 en=1;
38 reset= 1;
39 data = 1;
40 #30 reset =0;
41
42
43 #10 data =1;
44 #10 data =0;
45 #10 data =0;
46 #10 data =0;
47 #10 data =0;
48 #10 data =0;
49 #10 data =0;
50 #10 data =0;
51 #10 data =0;
52 #10 data =0;
53 #10 data =0;
54 #10 data =1;
55
56 #10 data =1;
57 #10 data =0;
58 #10 data =0;
59 #10 data =0;
60 #10 data =0;
61 #10 data =0;
62 #10 data =0;
63 #10 data =0;
64 #10 data =0;
65 #10 data =0;
66 #10 data =1;
67
68 #10 data =1;
69 #10 data =0;
```

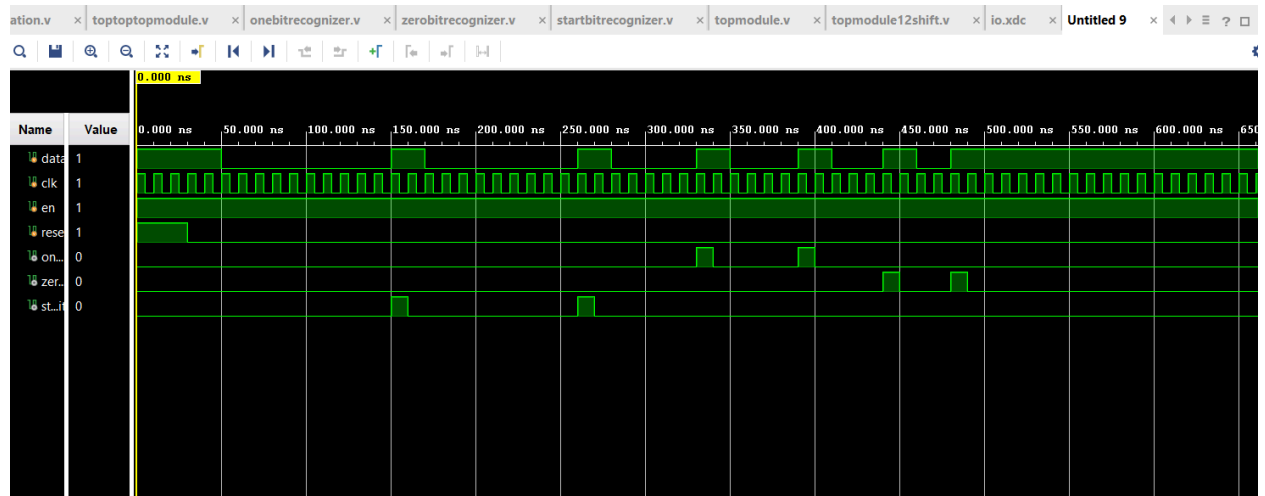
Fig 5 cont.

```
allbitsimulation.v x toptoptopmodule.v * x onebitrecognizer.v * x zerobitre
C:/Users/amshelton/Desktop/Prelab4_fixed/Prelab4_fixed.srscs/sim_1/new/allbitsimulation.v

52  ○ #10 data =0;
53  ○ #10 data =0;
54  ○ #10 data =1;
55
56  ○ #10 data =1;
57  ○ #10 data =0;
58  ○ #10 data =0;
59  ○ #10 data =0;
60  ○ #10 data =0;
61  ○ #10 data =0;
62  ○ #10 data =0;
63  ○ #10 data =0;
64  ○ #10 data =0;
65  ○ #10 data =0;
66  ○ #10 data =1;
67
68  ○ #10 data =1;
69  ○ #10 data =0;
70  ○ #10 data =0;
71  ○ #10 data =0;
72  ○ #10 data =0;
73  ○ #10 data =0;
74  ○ #10 data =1;
75
76  ○ #10 data =1;
77  ○ #10 data =0;
78  ○ #10 data =0;
79  ○ #10 data =0;
80  ○ #10 data =0;
81  ○ #10 data =1;
82
83  ○ #10 data =1;
84  ○ #10 data =0;
85  ○ #10 data =0;
86  ○ #10 data =0;
87  ○ #10 data =1;
88
89  ○ #10 data =1;
90  ○ #10 data =0;
91  ○ #10 data =0;
92  ○ #10 data =1;
93
94
95
96
97  end
98  endmodule
99
```

Fig 6: Timing Diagram

- simulating zero, one and start bit recognizer's working in tandem to classify data



Prelab Simulations and Verilog Code:

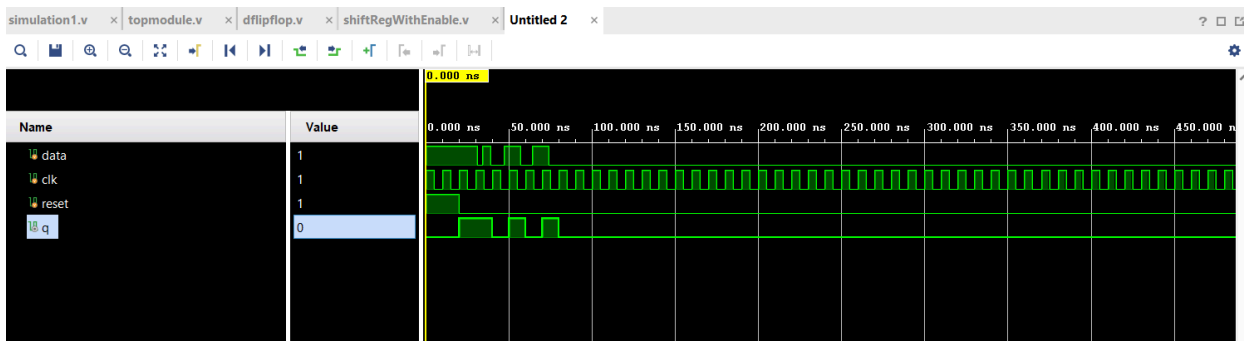


Fig 7: D Flip Flop Simulation

```

Project Summary x Schematic x simulation1.v x topmodule.v
C:/Users/amshelton/Desktop/Prelab4/Prelab4.srscs/sim_1/new/simulat

17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module simulation1();
24
25
26 reg data, clk, reset;
27 wire q;
28
29 topmodule a(data, clk, reset, q);
30 always #5 clk = ~clk;
31
32 initial
33 begin
34   clk = 1;
35   reset = 1;
36   data = 1;
37   #20 reset = 0;
38   #1 data = 1;
39   #10 data = 0;
40   #3 data = 1;
41   #5 data = 0;
42   #8 data = 1;
43   #10 data = 0;
44   #7 data = 1;
45   #10 data = 0;
46
47
48
49
50 end
51 endmodule
52

```

```

Project Summary x Schematic x simulation1.v x topmodule.v x dflipflop.v x s
C:/Users/amshelton/Desktop/Prelab4/Prelab4.srscs/sources_1/new/topmodule.v

1 `timescale 1ns / 1ps
2 ///////////////////////////////////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 11/05/2025 03:04:36 PM
7 // Design Name:
8 // Module Name: topmodule
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module topmodule( input data, input clk, input reset, output q);
24
25   dflipflop segment1(data, clk, reset, q);
26
27
28 endmodule
29

```

Fig 8+9: D Flip Flop Simulation (8) and Top module (9)

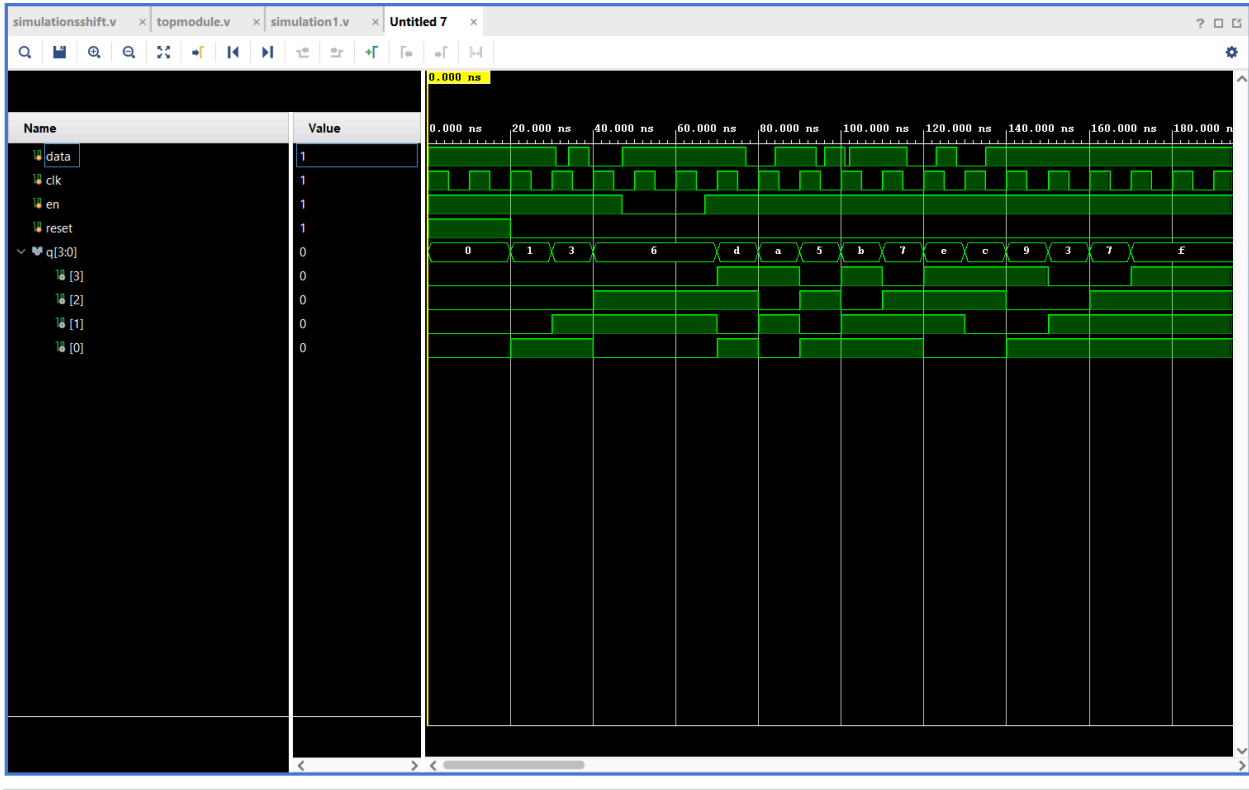


Fig 10: 4-bit Shift recognizer Simulation

```
simulationsshift.v x topmodule12shift.v x topmodule2.v x shiftRegWithEnable.v x Untitled 9 x
C:/Users/amshelton/Desktop/prelab4_moredone/prelab4_moredone/prelab4_moredone.srcs/sim_1/new/simulationsshift.v

19 //
20 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
21
22
23 module simulationsshift();
24
25 reg data, clk, en , reset;
26 wire [11:0]q;
27
28 topmodule12shift c(data, clk,en, reset, q);
29 always #5 clk = ~clk;
30
31 initial
32 begin
33
34 clk =1;
35 en=1;
36 reset= 1;
37 data = 1;
38 #20 reset =0;
39 #1 data =1;
40 #10 data =0;
41 #3 data =1;
42 #5 data =0;
43 #8 data =1;
44 en=0;
45 #20 en=1;
46 #10 data =0;
47 #7 data =1;
48 #10 data =0;
49 #1 data =0;
50 #1 data =1;
51 #5 data =0;
52 #1 data =1;
53 #10 data =1;
54 #4 data =0;
55 #7 data =1;
56 #5 data =0;
57 #7 data =1;
58 #10 data =1;
59
60
61
62
63 end
64 endmodule
65
66
```

Fig 11: Recognizer Simulation source

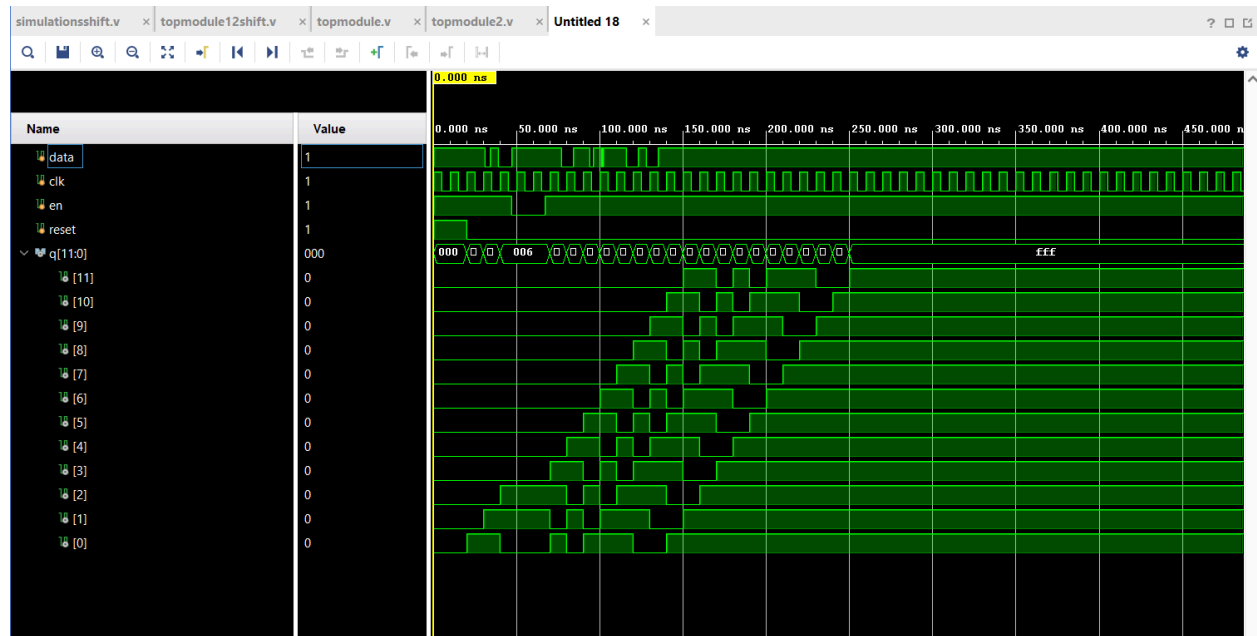
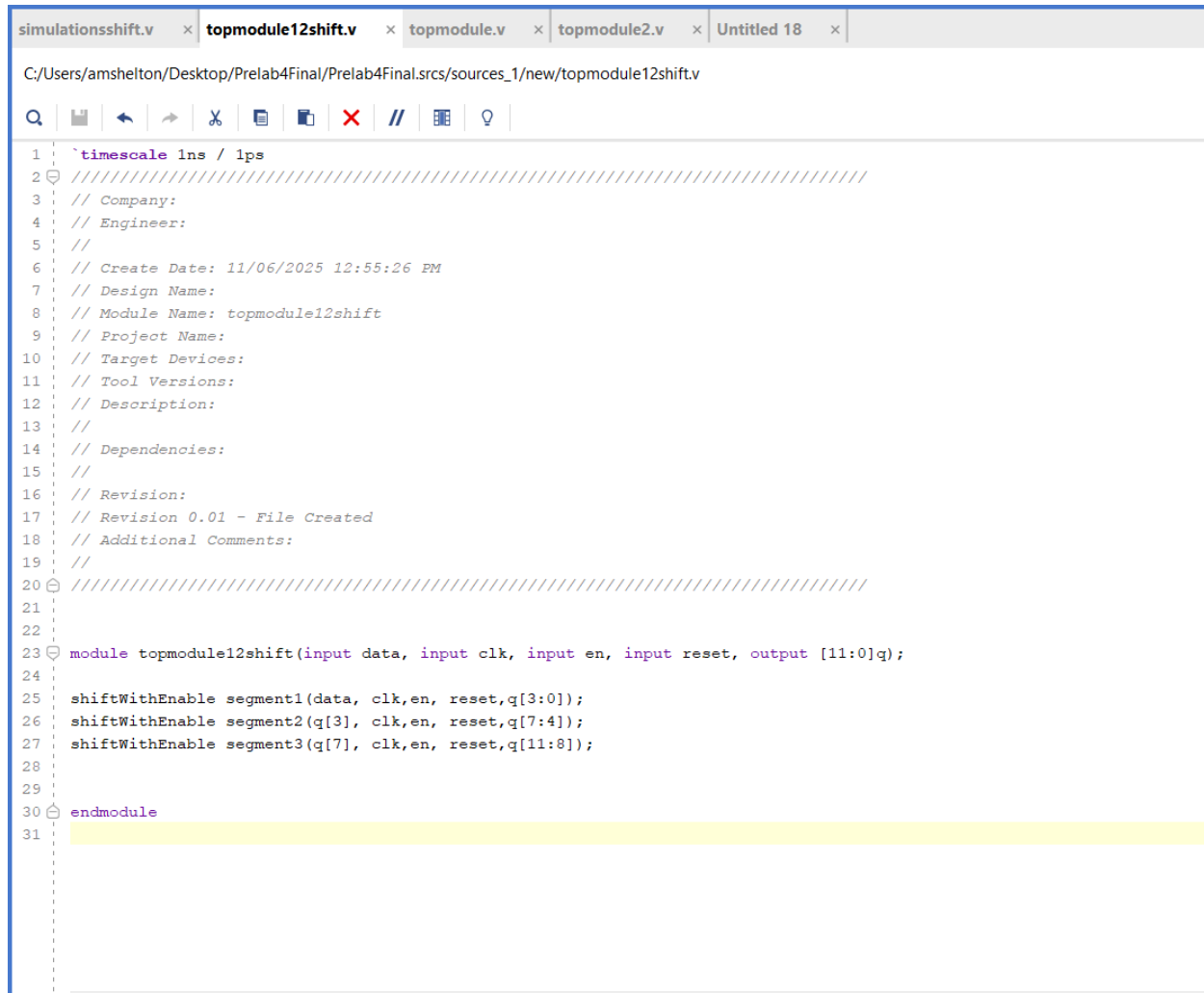


Fig 12: 12 bit recognizer simulation



```
1 `timescale 1ns / 1ps
2 //////////////////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 11/06/2025 12:55:26 PM
7 // Design Name:
8 // Module Name: topmodule12shift
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////////////////
21
22
23 module topmodule12shift(input data, input clk, input en, input reset, output [11:0]q);
24
25   shiftWithEnable segment1(data, clk,en, reset,q[3:0]);
26   shiftWithEnable segment2(q[3], clk,en, reset,q[7:4]);
27   shiftWithEnable segment3(q[7], clk,en, reset,q[11:8]);
28
29
30 endmodule
31
```

Fig 13: 12 bit shift tip module code.

Analysis

- The shift register works similarly to the D- flip flop, it is dependent on the clock rising and when the clock rises, the input is checked and the output can change. The shift register is different because there are multiple outputs that all depend on the previous one. If the input changes at a clock rising time, it makes a change in the first output, then when the output can change again, that value will shift upwards, essentially counting up.
- The purpose of an enabling input was to choose whether or not the decoder would interpret the signals being sent to it. With the enable input on the decoder classifies the data benign sent to it, with the enable off the decoder does nothing. Functionally it allowed us to turn the receiver on and off.

- We determined the number of 0's for each bit by first calculating the period = $1/\text{frequency}$. Then we divided the delay by the period. If the result was a decimal in between two numbers, the two numbers were used as different cases because there are 2 possible waveform alignment for each SONY bit.

The lab went accordingly, through our combined effort as a team. There were minor hiccups with implementation of the start bit pin on the FPGA not responding to given input signals, but through our tenacity as a team and the helpful guidance of lab TA's we were able to determine it was a hardware error stemming from the FPGA itself rather than any verilog error or circuit implementation.

Conclusion

In conclusion, we achieved our goal of better understanding shift registers and sequential circuits through implementing real life examples like using a remote. We also got better acquainted with the simulation software Verilog and using top modules to create larger, more complex circuits. This lab demonstrated the connection between software code and hardware implementation through the Basys and breadboard circuitry.